

Microservices Resilience, Observability and Monitoring

Introduzione

I tre argomenti

Argomento	Contenuto
1. Resilienza e fault tolerance	Retry, Circuit Breaker, Bulkhead, Timeout, Fallback
2. Osservabilità	Logging distribuito (Elastic Stack: Elasticsearch, Logstash, Kibana) + Tracing distribuito (OpenTelemetry + Zipkin)
3. Monitoring	Health monitoring (Watchdog, Kubernetes health checks, Prometheus, Grafana)

Il problema ereditato

*"La nostra attività di e-commerce sta crescendo con **traffico in forte aumento**. Dovremmo gestire **milioni di richieste con latenza accettabile**, ma in alcuni casi, a causa del traffico intenso, i microservizi hanno subito eccezioni e hanno interrotto casi d'uso importanti, causando perdita di fatturato e mettendo in crisi la fedeltà dei clienti."*

Le esigenze:

- Distribuire nuove funzionalità senza influenzare il sistema
- "Se qualcosa va storto, **il sistema deve RIPRENDERSI dai guasti**"
- "Ad esempio si verificano **errori transitori** come interruzioni del server o problemi di rete. Questo potrebbe essere **l'auto-guarigione e il recupero dai guasti**"

★ "I microservizi devono essere **progettati per essere resilienti**, perché i sistemi di microservizi sono costituiti da **molti componenti indipendenti che comunicano attraverso CONNESSIONI DI RETE**, e qualsiasi problema con questi componenti o con le connessioni può avere un impatto sul sistema nel suo complesso."

What is Microservices Resiliency

★ Il cambio di mentalità

*"Dobbiamo **PRESUMERE** che i guasti si verificheranno. Le applicazioni a microservizi **prima o poi falliranno**, e noi dobbiamo **ACCETTARE i fallimenti**."*

★ "NON dovremmo progettare i sistemi con il presupposto che funzioneranno senza problemi per anni. Dovremmo progettare microservizi con i guasti, preparati per gli scenari peggiori."

Questo è il capovolgimento mentale della lezione: nel monolite un guasto era un'eccezione; **nel distribuito è la normalità statistica.**

Le cause di fallimento

*"Un singolo microservizio può fallire o non essere disponibile **per un breve periodo**. Poiché **client e servizi sono processi SEPARATI**, un servizio potrebbe:"*

- essere **sovraccarico**
- **rispondere molto lentamente**
- **non essere accessibile** per problemi di rete

★ Il fallimento a cascata

"Se il servizio A chiama il servizio B, che a sua volta chiama il servizio C — cosa succede quando B è inattivo? Qual è il vostro piano di ripiego?"

L'esempio del corso:

[API Gateway] → [Basket] → [Discount] → [Ordering] → risposta

✗ DOWN

"Cosa succede se il microservizio Discount non funziona? Come possiamo continuare a rendere il nostro sistema operativo e funzionale?"

Definizione

La resilienza dei microservizi è la capacità di un sistema di RESISTERE e RIPRENDERSI da guasti o interruzioni.

*"Un microservizio deve essere resiliente ai guasti e **deve ACCETTARE I GUASTI PARZIALI**. Dovrebbero essere in grado di **riprendersi dai guasti e continuare a funzionare**. Dovremmo accettare i guasti e reagire ad essi senza alcun downtime o perdita di dati."*

★ *"Lo scopo principale della resilienza è riportare l'applicazione a uno stato **PIENAMENTE FUNZIONALE** dopo un guasto."*

Definizione: guasto parziale (partial failure) — la situazione tipica dei sistemi distribuiti in cui *alcune* parti funzionano e altre no. Nel monolite non esisteva: o il processo era su, o era giù. Nei microservizi, **la maggior parte dei guasti sono parziali**, ed è per questo che si possono gestire con grazia.

Microservices Resiliency Patterns

Perché è difficile

*"Garantire la durabilità nelle architetture a microservizi è **relativamente difficile** rispetto alle architetture monolitiche. La comunicazione tra i servizi è **distribuita** e per **UNA transazione viene creato MOLTO traffico di rete interno o esterno**."*

★ *"Con l'aumento della necessità di comunicazione tra i servizi e della dipendenza da servizi esterni, AUMENTERÀ LA POSSIBILITÀ CHE SI VERIFICHINO ERRORI."*

★ I 6 pattern in sintesi

Pattern	Cosa fa
Retry	<i>"Riprovare una richiesta se fallisce o va a vuoto. Può essere implementato a livello di client o di servizio. Gestisce guasti o interruzioni TEMPORANEE"</i>
Circuit Breaker	<i>"Introdurre un proxy tra client e servizio. Controlla lo stato di salute del servizio e, se rileva che non è disponibile, APRE il circuito e IMPEDISCE l'invio di ulteriori richieste"</i>
Bulkhead	<i>"Suddividere un sistema in componenti ISOLATI per evitare che il guasto di un componente si ripercuota sugli altri"</i>
Timeout	<i>"Non attendere la risposta di un servizio per un tempo INDEFINITO: lanciare un'eccezione invece di aspettare"</i>
Fallback	<i>"Fornire un comportamento o una risposta ALTERNATIVI se una richiesta fallisce. Es. una versione in cache dei dati, o un messaggio predefinito"</i>
Cache	<i>"Memorizza le risposte" — già visto nella Lezione 13</i>

Retry Pattern

Il problema che risolve

"La comunicazione interservizi può avvenire tramite HTTP o gRPC. Le interruzioni fisiche possono verificarsi. Se uno dei servizi restituisce HTTP 500 durante la transazione, questa può essere interrotta e all'utente può essere mostrato un errore."

★ *"Ma quando l'utente riavvia la stessa operazione, spesso funziona. Sarebbe più logico RIPETERE la richiesta al servizio che ha restituito 500, in modo che i rollback delle transazioni già realizzate non debbano funzionare e l'utente possa eseguire la transazione senza accorgersene."*

Definizione

Un design pattern che consente a un sistema di RIPROVARE automaticamente un'operazione fallita.

"Le comunicazioni possono fallire a causa di ERRORI TRANSITORI, ma questi guasti si verificano in breve tempo e possono risolversi da soli."

Definizione: errore transitorio (transient fault) — un errore temporaneo e auto-risolvente: un pacchetto perso, un timeout di rete momentaneo, un breve sovraccarico. La caratteristica chiave è che ritentare ha buone probabilità di successo.

Il caso d'uso

"Un servizio carrello che chiama un servizio di pagamento. Se il pagamento è temporaneamente non disponibile per un problema di rete, il carrello può usare il Retry per riprovare automaticamente la richiesta."

L'esempio del flusso

Client → [API Gateway] → [Microservizio interno]

```
| 1° tentativo → HTTP 500 ✗  
| 2° tentativo → HTTP 500 ✗  
L 3° tentativo → SUCCEED ✓
```

"L'API Gateway riceve due volte un errore. E infine, alla TERZA chiamata, ha ottenuto "Succeed". La transazione è stata completata con successo SENZA terminare [con errore per l'utente]."

★ L'exponential back-off

"È importante PROLUNGARE il tempo di intervallo tra i tentativi."

Definizione: back-off time — "gli intervalli di tempo tra le richieste ripetute".

★ *"Nella maggior parte degli usi, il periodo di back-off dovrebbe essere un RITIRO ESPONENZIALE INCREMENTALE, per consentire [al servizio di riprendersi]."*

1° retry → attendi 1s

2° retry → attendi 2s

3° retry → attendi 4s

4° retry → attendi 8s

Perché è cruciale: se il servizio è sovraccarico e tutti i client ritentano **immediatamente e insieme**, il retry **peggiora** il sovraccarico. L'exponential back-off dà al servizio il tempo di riprendersi, e distribuisce i tentativi nel tempo.

⚠ Il limite del Retry

★ *"Se un servizio presenta guasti PERSISTENTI o è indisponibile per un periodo PROLUNGATO, il Retry può causare [più danni]. Questo CONSUMERÀ LE NOSTRE RISORSE."*

"In questi casi potrebbe essere necessario una logica di Circuit Breaker o di Fallback per evitare [il problema]."

Circuit Breaker Pattern

L'origine del nome

*"Un metodo nei circuiti elettronici: gli interruttori **INTERROMPONO** il trasferimento del carico in caso di guasto nel sistema, **PER PROTEGGERE** [il resto del circuito]."*

Definizione

Il Circuit Breaker protegge dai guasti in cui un guasto in un servizio può avere EFFETTI A CASCATA sugli altri.

*"Se un microservizio dipende da un secondo microservizio e **questo fallisce**, anche il primo **fallirebbe**. Tuttavia, se il primo usa un **Circuit Breaker**, può **PREVENIRE** ulteriori chiamate al secondo microservizio e **CONTINUARE A FUNZIONARE**, anche se il secondo è down."*

L'esempio

*"Un **microservizio di prenotazione** dipende da un **servizio esterno** per le informazioni sui voli. Se il servizio esterno diventa **indisponibile**, anche il **microservizio di prenotazione fallirebbe**. Implementando un **Circuit Breaker**, se il servizio esterno diventa **indisponibile**, l'interruttore **SCATTA**, impedendo ulteriori chiamate."*

Il meccanismo

*"Il **Circuit Breaker** **MONITORA** la comunicazione tra i servizi e traccia gli errori. Quando l'errore supera un certo **VALORE DI SOGLIA**, l'interruttore si attiva e **INTERROMPE** la comunicazione, restituendo [direttamente l'errore precedente]."*

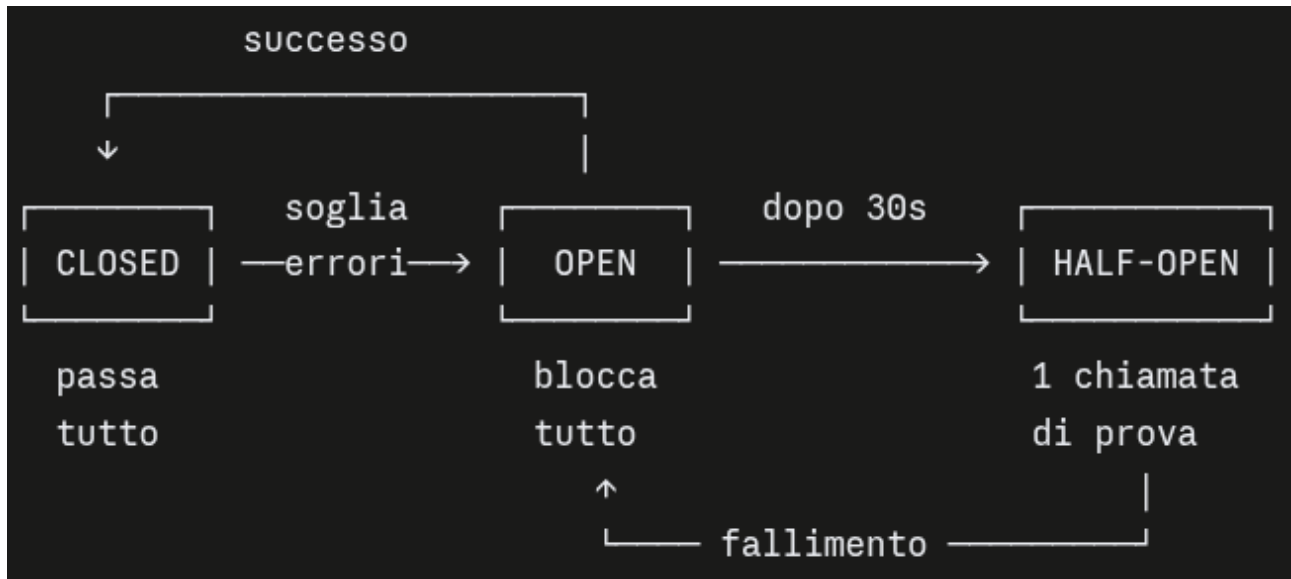
★ *"Continua a monitorare il traffico e, se il servizio richiesto inizia [a rispondere], [richiude il circuito]."*

L'esempio concreto del corso

1. [API Gateway] → [Basket] → errore, errore, errore...
2. SOGLIA SUPERATA → ⚡ IL CIRCUITO SI APRE
3. Le richieste successive NON raggiungono più Basket
→ viene restituita DIRETTAMENTE l'eccezione ottenuta in precedenza
4. 🕒 Ogni 30 secondi, il Circuit Breaker CONSENTE UNA CHIAMATA per controllare lo stato del microservizio
5. Se risponde → il circuito si CHIUDE e il traffico riprende

Il vantaggio che spesso sfugge: quando il circuito è aperto, **il client riceve l'errore Istantaneamente** invece di aspettare un timeout. Questo **libera i thread del chiamante** — che è esattamente ciò che impedisce la cascata.

Circuit Breaker Status



Stato	Comportamento
CLOSED (chiuso)	"L'interruttore NON è aperto" — le richieste passano normalmente al servizio
OPEN (aperto)	★ "Impedisce all'applicazione di tentare ripetutamente [la chiamata]. Se l'interruttore è aperto, NON si tratta di una richiesta diretta ai microservizi: si tratta fondamentalmente di rispondere con l'eccezione "
HALF-OPEN (semiaperto)	"L'interruttore esegue ALCUNE operazioni per IDENTIFICARE se si verifica ancora un errore dal servizio "

Lo stato **HALF-OPEN** è il più interessante: è il "tentativo cauto". Se la chiamata di prova riesce → **CLOSED** (tutto torna normale). Se fallisce → **di nuovo OPEN** per altri 30 secondi. Impedisce di riaprire il traffico prematuramente e riaffondare il servizio appena ripartito.

Retry + Circuit Breaker Pattern

★ Perché il solo Retry è pericoloso

"Cosa succede se il microservizio interno è **TOTALMENTE INATTIVO** e non può essere pronto per molto tempo?"

★ "Se continuiamo a ripetere i tentativi verso un servizio che non risponde, questo **CONSUMERÀ I THREAD** e le **CONNESSIONI AL DATABASE**, e può causare [il collasso del chiamante]. È quindi **PERICOLOSO** seguire solo il pattern **Retry**."

La combinazione

```
Client → [API Gateway]
      |
      ├── RETRY: riprova 3 volte con back-off esponenziale
      |           (gestisce gli errori TRANSITORI)
      |
      └── dopo 10 richieste fallite:
          CIRCUIT BREAKER si apre
          (gestisce gli errori PERSISTENTI)
              ↓
          ogni 30s → 1 chiamata di prova
              ├── riesce → circuito CHIUSO, traffico ripristinato
              └── fallisce → circuito resta APERTO
```

★ La distinzione fondamentale

"Il pattern Retry riprova SOLO un'operazione [che ci si aspetta possa riuscire]."

"Il Circuit Breaker IMPEDISCE che la comunicazione venga tentata ripetutamente, quando è probabile che fallisca."

	Retry	Circuit Breaker
Gestisce	Errori transitori (brevi)	Errori persistenti (prolungati)
Assunzione	"probabilmente al prossimo tentativo funziona"	"probabilmente è rotto, smetti di provare"
Effetto	Aumenta le chiamate	Riduce le chiamate
Protegge	L'utente dall'errore momentaneo	★ Il chiamante dal consumo di risorse e il servizio a valle dal sovraccarico

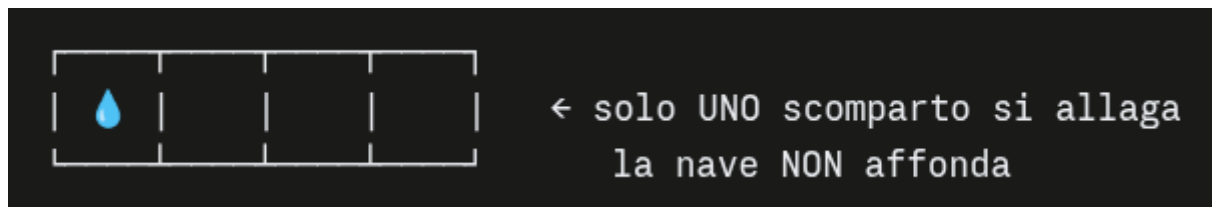
*"Nella nostra architettura **applicheremo ENTRAMBI**: quando il client invia la richiesta, [si applica il Retry]. E poi, se i **microservizi a valle NON rispondono**, dobbiamo applicare il Circuit Breaker."*

Bulkhead Pattern

L'origine del nome

*"Si basa sul concetto di **paratia in una NAVE**: una partizione strutturale che aiuta a prevenire l'**ALLAGAMENTO** [dell'intero scafo]."*

*"Quando si costruiscono navi o sottomarini, questi **NON** vengono realizzati [come un unico volume]. [Se uno scomparto si allaga,] il **compartimento interessato viene CHIUSO E ISOLATO**."*



*"Se pensiamo ai nostri microservizi come a una nave, **dobbiamo essere in grado di rispondere alle richieste degli utenti ISOLANDO i servizi pertinenti**, in modo che **l'errore che si verifica in un servizio NON influisca sugli altri**."*

★ Il problema specifico: l'esaurimento del thread pool

Questo è il meccanismo che il bulkhead risolve, e va capito bene:

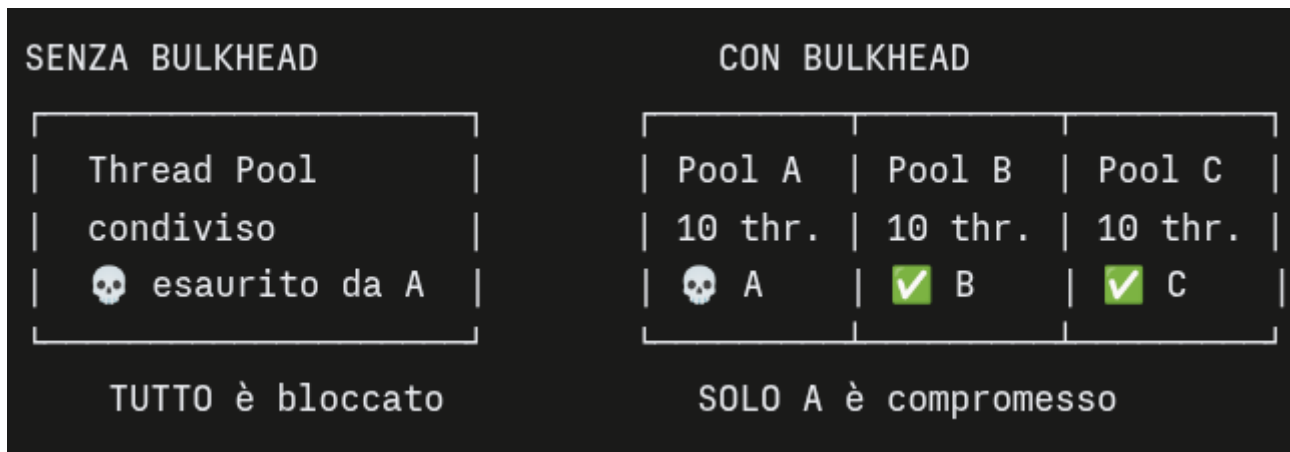
1. Gli utenti fanno richieste al Servizio A
2. Il Servizio A **NON RISPONDE** (bug software)
3. Le richieste degli utenti si **ACCUMULANO** in attesa
4. Dopo un po', le richieste usano **TUTTI** i thread CPU della macchina
5. Tutte le **NUOVE** richieste iniziano [a essere rifiutate]
6. ⚠ Mentre le richieste al Servizio A (**ROTTO**) occupano tutte le risorse, le richieste al Servizio B (che **FUNZIONA**) continuano ad **ASPETTARE**

"Mentre TUTTE le richieste al servizio A che non funziona sono occupate dalle risorse, le richieste al servizio B CHE FUNZIONA CON SUCCESSO continueranno ad ASPETTARE."

Questo è il punto cruciale: il servizio rotto **affama** quello sano, perché condividono lo stesso pool di risorse.

La soluzione

"Il Bulkhead suggerisce che, quando un servizio non risponde, INVECE di usare TUTTE le risorse per elaborare la richiesta a quel servizio, ALCUNE risorse vengono allocate in un PROCESSO SEPARATO, e le altre richieste vengono elaborate con le risorse [rimanenti]."



Come si implementa

"È possibile definire **LIMITI DI RISORSE** per il server applicativo: ad esempio il **NUMERO MASSIMO DI THREAD** e [la memoria]."

L'esempio e-commerce

"Un'applicazione di e-commerce che **SEPARA le istanze di servizio** per la gestione del carrello, l'elaborazione dei pagamenti, la gestione del catalogo. [Se il pagamento si guasta,] il Bulkhead **isola** [quel guasto], in modo che **il servizio di carrello e catalogo CONTINUINO** a funzionare."

Quando usarlo — i 4 scenari

1. "Quando **un servizio ha un carico elevato** e c'è **rischio di contesa di risorse**"
2. "Quando un servizio **dipende da uno o più servizi a valle** e c'è rischio [di propagazione del guasto]"
3. "Se **un servizio a valle diventa indisponibile**"
4. "Quando un servizio deve effettuare **chiamate frequenti a un servizio a valle** e c'è **rischio di sovraccaricarlo**"

Timeout Pattern

Definizione

Non si dovrebbe attendere la risposta di un servizio per un tempo **INDEFINITO**: si dovrebbe lanciare un'eccezione invece di aspettare.

"Si tratta di **impostare un LIMITE DI TEMPO MASSIMO** per il completamento. Se il limite viene superato, la chiamata viene considerata [fallita]."

L'esempio

"Un'applicazione e-commerce con **un servizio carrello e un servizio pagamento separato**. Se il pagamento impiega più tempo del previsto, il Timeout **ANNULLA la richiesta e restituisce un errore al microservizio carrello**, anziché permettere che la richiesta [resti appesa]."

★ Il collegamento con l'esperienza utente

"Questo è **importante per l'esperienza del cliente**: in questo caso abbiamo impostato il tempo su **30 secondi**, il che significa che **il cliente attende AL MASSIMO 30 secondi** se il pagamento [non risponde]."

★ "Altrimenti, se non si applicano i pattern di timeout, il cliente aspetta per un tempo **INDEFINITO**, e questo sarà [dannoso] per la fidelizzazione e l'esperienza [utente]."

Nota architetturale: il Timeout è il pattern **abilitante** degli altri. Senza timeout, il Circuit Breaker non "vede mai" un fallimento (la chiamata non torna), e il Bulkhead si riempie comunque. **Il timeout è ciò che trasforma "lento" in "fallito"**, e solo allora gli altri pattern possono agire.

Fallback Pattern

Definizione

Fornisce un **COMPORTAMENTO** o una **RISPOSTA ALTERNATIVA** se una richiesta fallisce o va in timeout.

"Si tratta di **definire una FUNZIONE DI FALLBACK** che viene richiamata se la chiamata fallisce o va in timeout. La funzione fornisce **una risposta** [alternativa]."

Gli esempi

"Se un servizio non è disponibile, il client può:"

- "usare **una versione IN CACHE dei dati**"
- "visualizzare **un messaggio predefinito**"

"Se il servizio di elaborazione pagamenti non è disponibile, il Fallback può essere usato **per visualizzare un messaggio di errore o offrire** [un'alternativa]."

"Questo tipo di **risposta predefinita** viene restituita **all'applicazione client**."

Perché il Fallback è la coppia naturale del Circuit Breaker: quando il circuito è aperto, **devi restituire qualcosa**. Il Fallback è quel "qualcosa". Insieme formano: "non riesco a chiamare Discount → non blocco l'utente → mostro il prezzo senza sconto e lo dico". **Degrado grazioso invece di errore**.

Riepilogo dei 5 pattern di resilienza

Pattern	Quando interviene	Cosa protegge
Timeout	La chiamata è lenta	Trasforma "lento" in "fallito" — abilita tutto il resto
Retry	La chiamata è fallita transitoriamente	L' utente da un errore momentaneo
Circuit	Le chiamate falliscono	Il chiamante (thread) e il chiamato

Pattern	Quando interviene	Cosa protegge
Breaker	ripetutamente	(sovraccarico)
Bulkhead	Un servizio satura le risorse	Gli altri servizi che condividono le risorse
Fallback	Tutto quanto sopra è fallito	L' esperienza utente — degrado grazioso
Insieme formano una catena logica: Timeout rileva → Retry tenta → Circuit Breaker rinuncia → Fallback degrada → Bulkhead contiene.		

Observability with Distributed Logging and Tracing

Definizione

L'osservabilità dei microservizi è la capacità di **MONITORARE e COMPRENDERE** il comportamento e le prestazioni [di un sistema distribuito].

"Uno dei modi migliori per comprendere e gestire questa complessità è l'osservabilità. Ci offre una maggiore visione operativa e ci porta a comprendere gli incidenti."

I 2 strumenti chiave

1. Distributed Logging

*"La pratica di **RACCOGLIERE, ARCHIVIARE e ANALIZZARE** i dati di log da **PIÙ ISTANZE DI SERVIZIO**."*

*"Utile per **comprendere il comportamento del sistema NEL TEMPO** e **identificare** [problemi e pattern]."*

2. Distributed Tracing

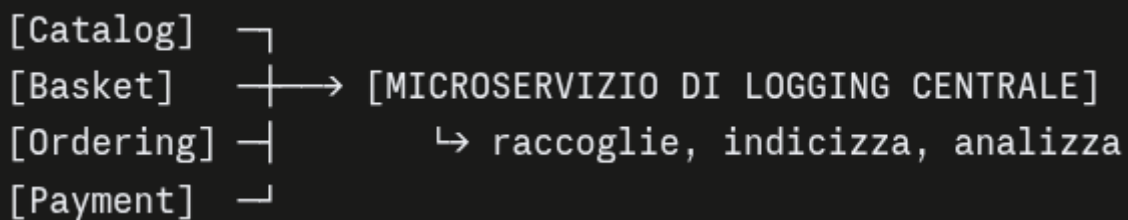
*"La pratica di **TRACCIARE IL FLUSSO DI UNA RICHIESTA** attraverso l'**architettura**, per capire come le diverse istanze interagiscono."*

*"Utile per **comprendere le prestazioni** e **identificare i COLLI DI BOTTIGLIA**."*

★ La differenza fondamentale

	Logging	Tracing
Cosa cattura	Eventi in un singolo servizio	Il percorso di una richiesta attraverso N servizi
Risponde a	"Cosa è successo dentro il servizio X?"	"Dove è andata questa richiesta, e quanto ha impiegato in ogni tappa?"
Asse	Temporale (nel tempo)	Spaziale (attraverso i servizi)

L'architettura



"Ogni microservizio effettua il logging nel microservizio di logging CENTRALE. Questi rappresenteranno i log e identificheranno [i problemi]."

★ Correlation ID e Trace ID

*"Quando un'applicazione client attiva il caso d'uso **"Aggiungi un articolo al carrello"**, questo verrà attivato a valle nel microservizio Basket, e questo servizio potrebbe [chiamare] il servizio di pagamento e ordinazione."*

★ *"Possiamo usare il **CorrelationID** o **TraceID** per identificare **quale passo ha ricevuto** [l'errore]. Possiamo facilmente identificare il microservizio [in cui si è verificato il problema] **SEGUENDO il CorrelationID.**"*

Definizione: Correlation ID — un identificatore univoco generato all'ingresso della richiesta (tipicamente dall'API Gateway) e **propagato in ogni chiamata successiva**, attraverso gli header HTTP o i metadati dei messaggi. Permette di **filtrare tutti i log di TUTTI i servizi per una singola richiesta utente** — è ciò che ricostruisce lo stack trace perduto nel mondo distribuito.

Elastic Stack for Distributed Logging

Definizione

Una raccolta di strumenti OPEN SOURCE per la raccolta, l'archiviazione e l'analisi dei dati di log.

*"Fornisce **una piattaforma flessibile e scalabile** per monitorare e comprendere il comportamento e le prestazioni* [dei microservizi]."**

★ I 4 componenti

Componente	Ruolo
Elasticsearch	<i>"Motore di ricerca e analisi DISTRIBUITO che può essere usato per archiviare [e cercare i log]"</i>
Logstash	<i>"Strumento di RACCOLTA e TRASFORMAZIONE dei dati"</i>
Kibana	<i>"Strumento di VISUALIZZAZIONE ed esplorazione dei dati, per creare dashboard"</i>
Beats	<i>"Una raccolta di CARICATORI DI DATI LEGGERI (shipper) che raccolgono log e altri dati da diverse fonti e li inviano" [a Logstash o Elasticsearch]</i>

Il flusso

[Microservizi] → log

↓ (Beats o direttamente)

[LOGSTASH] ← raccoglie e TRASFORMA

↓

[ELASTICSEARCH] ← indicizza e ARCHIVIA (indici ricercabili)

↓

[KIBANA] ← VISUALIZZA: dashboard, ricerca, analisi

*"I log vengono **raccolti e trasformati con Logstash**, poi inviati a **Elasticsearch**, dove possiamo **cercare e analizzare creando indici**. Dopodiché **Elasticsearch** può essere **visualizzato e gestito con Kibana** per vedere e **identificare pattern e tendenze**."*

I casi d'uso

Root cause analysis ★:

*"Se il servizio di elaborazione pagamenti presenta un **TASSO DI ERRORE ELEVATO**, Elastic Stack può essere usato **per IDENTIFICARE LA CAUSA PRINCIPALE** del problema e suggerire potenziali [soluzioni]. Osservando e analizzando i log è possibile **identificare la causa principale**."*

Alerting:

*"Elastic Stack può essere usato per **impostare AVVISI o NOTIFICHE** per avvisare il team quando vengono rilevati determinati eventi o condizioni."*

Performance monitoring:

*"Monitorare i **tempi di risposta e i tassi di errore** dei microservizi, o **tracciare il flusso delle richieste**. Questo può aiutare a **identificare i colli di bottiglia**."*

Distributed Tracing with OpenTelemetry using Zipkin

Definizione: OpenTelemetry

Un progetto open source che fornisce un insieme di API e LIBRERIE per la **RACCOLTA e l'ESPORTAZIONE** di dati di telemetria — tracce, metriche e log — compresi i dati per il tracing distribuito.

Nota: OpenTelemetry è uno **standard**, non un backend. Definisce *come* strumentare il codice; poi i dati possono essere esportati verso Zipkin, Jaeger, Datadog, ecc. Il valore è che **non ti legghi a un vendor**: cambi l'exporter, non il codice.

Definizione: Zipkin

Un sistema di tracing distribuito [che riceve, archivia e visualizza le tracce].

Come funziona insieme

1. L'SDK OpenTelemetry è **INTEGRATO** in **OGNI** istanza di servizio
2. I servizi **RACCOLGONO** i dati di tracing mentre le richieste fluiscono
3. I dati vengono **INVIATI** al server Zipkin per l'archiviazione
4. La Zipkin UI **VISUALIZZA** e analizza i dati di tracing

"Questo consentirebbe alle istanze **di raccogliere i dati di tracciamento MENTRE LE RICHIESTE FLUISCONO ATTRAVERSO IL SISTEMA** e di inviarli al server Zipkin."

L'esempio

"Se un utente aggiunge un articolo al carrello, **la traccia potrebbe includere informazioni sulla richiesta che passa DAL servizio carrello AL servizio pagamenti AL servizio catalogo.**"

★ "[La traccia] è illustrata molto bene: **quali API invocano quali microservizi, e quali microservizi [sono coinvolti].**"

"Usare OpenTelemetry con Zipkin **può contribuire a migliorare l'osservabilità, fornendo UN MODO STANDARD per raccogliere e visualizzare i dati di tracciamento.**"

Nota che questo è esattamente ciò che abbiamo visto nella dashboard .NET Aspire per tutto il corso — le tracce con i tempi per ogni segmento. Aspire usa proprio OpenTelemetry sotto il cofano.

Health Checks: Liveness, Readiness and Performance

Definizione

I controlli di salute sono un modo per MONITORARE [lo stato dei microservizi].

"Possono essere usati per **determinare se un microservizio funziona correttamente.**"

"Questo è importante perché **il guasto di un SINGOLO microservizio può avere [effetti sull'intero sistema].**"

★ I 3 tipi di health check

1. Liveness Check (controllo di vivacità)

"Se un controllo di liveness **FALLISCE**, può indicare che il microservizio **È ANDATO IN CRASH o HA SMESSO [di funzionare].**"

Domanda: "il processo è vivo?"

Azione se fallisce: riavviare il container.

2. Readiness Check (controllo di prontezza)

"Se un controllo di readiness FALLISCE, può indicare che il microservizio NON È ANCORA PRONTO A GESTIRE IL TRAFFICO, o che ha qualche problema."

★ *"Il microservizio FUNZIONA, ma in qualche modo NON È IN GRADO di gestire alcune richieste."*

Domanda: "il processo è pronto a ricevere traffico?"

Azione se fallisce: togliere dal load balancer (ma NON riavviare).

Perché la distinzione conta enormemente: un servizio che sta caricando una cache all'avvio è **vivo ma non pronto**. Se usassi solo il liveness, Kubernetes gli manderebbe traffico troppo presto. Se usassi il readiness come liveness, lo riavvieresti in loop.

Servono entrambi, e fanno cose diverse.

3. Performance Check (controllo delle prestazioni)

*"Consentono di **monitorare le prestazioni**, ad esempio **i tempi di risposta**."*

*"Se il risultato della verifica [supera una soglia], può indicare che il microservizio ha dei **PROBLEMI**, o che c'è un problema nel sistema che causa i problemi di [prestazione]."*

★ La sfumatura importante

*"Un microservizio **potrebbe essere attualmente** [vivo ma non sano]. **L'approccio migliore è fare PRIMA UN'INDAGINE**" — cioè non reagire automaticamente a ogni segnale, ma capire.*

*"Possiamo utilizzare **gli eventi di salute di un microservizio** che ci aiutano a **prendere DECISIONI INFORMATE**."*

Health Monitoring with Kubernetes, Prometheus and Grafana

Le probe di Kubernetes

*"Kubernetes mette a disposizione sonde di **LIVENESS** e **READINESS**."*

Probe	Cosa controlla	Azione
Liveness probe	<i>"Se un microservizio è ancora IN ESECUZIONE"</i>	Riavvia il pod
Readiness probe	Se è pronto a ricevere traffico	Rimuove dal Service (load balancer)
<i>"Se un probe fallisce, Kubernetes può RIAVVIARE o RIMUOVERE automaticamente il microservizio per mantenere il sistema [sano]."</i>		

Questo collega direttamente al **self-healing** della Lezione 14: le probe sono il **meccanismo** con cui Kubernetes sa quando intervenire.

Gli strumenti di monitoring

Strumento	Ruolo
Prometheus	<i>"Sistema di monitoraggio open source per raccogliere e memorizzare [metriche]."</i>

Strumento	Ruolo
★	Fornisce un potente LINGUAGGIO DI INTERROGAZIONE
Grafana	"Piattaforma di VISUALIZZAZIONE dei dati e monitoraggio, usata insieme a Prometheus per creare DASHBOARD e visualizzazioni "
Datadog	"Piattaforma di monitoraggio e analisi basata sul cloud "
La divisione del lavoro: Prometheus raccoglie e interroga (con PromQL), Grafana disegna. È la coppia standard dell'ecosistema Kubernetes.	

Log analysis

"Gli strumenti di analisi dei log possono essere usati per **analizzare i messaggi** [di log] e **identificare problemi**."

Strumenti: Elastic Stack, oppure **Fluentd** e **Splunk**.

Alerting

"L'impostazione di **AVVISI** e **NOTIFICHE** può contribuire a garantire [una risposta rapida ai problemi]. Possiamo usare **Slack, Teams, e-mail e SMS** per identificare e notificare."

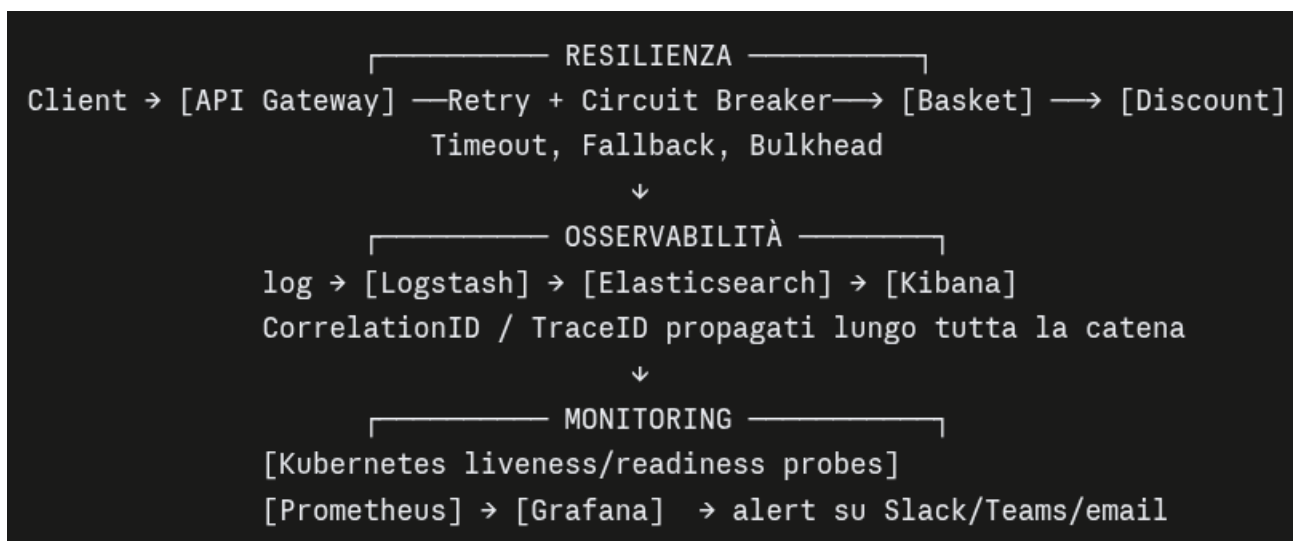
Design: Microservices Resilience, Observability and Monitoring

Toolkit aggiornato

Nuovi pattern e strumenti:

- **Resilienza:** Retry, Circuit Breaker, Bulkhead, Timeout, Fallback
- **Osservabilità:** Elastic Stack (Elasticsearch, Logstash, Kibana) + **tracing distribuito**
- **Monitoring:** Watchdog o Kubernetes Health Monitoring con **Prometheus e Grafana**

★ L'architettura finale



Cosa si applica dove:

"Qui si tratta dell'implementazione dello **schema di Retry e anche dello schema di Circuit Breaker**. Si tratta della **resilienza e della fault tolerance dei microservizi, delle best practice e dei pattern, integrati con i casi d'uso del carrello della spesa.**"

"Inoltre, stiamo seguendo l'**osservabilità dei microservizi**: qui l'implementazione di **Elastic Stack** per l'**AGGREGAZIONE DEI LOG** per analizzare le prestazioni."

★ "Allo stesso modo abbiamo applicato **CorrelationID e TraceID** per fornire il tracciamento distribuito. **Con questi seguiamo fondamentalmente il flusso della richiesta e IDENTIFICHIAMO QUALE MICROSERVIZIO HA AVUTO UN'ECCEZIONE.**"

"Infine, il monitoraggio: **sonde Kubernetes liveness e readiness** e strumenti come **Prometheus e Grafana.**"

I 4 vantaggi dell'architettura resiliente

Vantaggio	Dettaglio
Migliore fault tolerance	"I pattern di resilienza aiutano un sistema a CONTINUARE A FUNZIONARE anche in presenza di guasti . Particolarmente importante nei sistemi mission-critical , dove i tempi* [di fermo sono inaccettabili]*"
Maggiore disponibilità	"Con Circuit Breaker e meccanismi di Fallback , è possibile garantire che il sistema rimanga disponibile ANCHE QUANDO [alcuni componenti falliscono]. Contribuisce a ridurre i tempi di inattività "
Migliore affidabilità	"Quando un sistema è progettato per essere resiliente, è MENO PROBABILE che si verifichino guasti o interruzioni "
Scalabilità migliorata ★	"Lo schema Bulkhead può consentire di SCALARE DIVERSI COMPONENTI IN MODO INDIPENDENTE , senza influenzare gli altri"

Adapt: lo stack tecnologico

La tabella finale

Categoria	Strumenti self-hosted	Opzioni cloud
Pattern di resilienza	Retry, Circuit Breaker, Bulkhead, Timeout, Fallback applicati "all'API Gateway e ai microservizi a valle"	—
Logging distribuito	Elastic Stack (Elasticsearch, Logstash, Kibana), Fluentd, Splunk	Amazon Elastic Cloud, Azure Elastic
Tracing distribuito	OpenTelemetry, Zipkin	AWS X-Ray
Health monitoring	Kubernetes health checks (liveness/readiness probes), Prometheus, Grafana, Datadog	Azure Application Insights, Amazon CloudWatch

"Questi sono **esempi** di stack tecnologici, ma **naturalmente è possibile cambiare e adottare stack diversi in base alle esigenze del progetto**. Sto solo dando un esempio di tecnologie **usate principalmente nell'industria del software.**"

